

Terraform Scenario-Based Questions

Scenario 1: Terraform State File is Deleted

Scenario

Your team stores the Terraform state locally. A developer accidentally deletes the `terraform.tfstate` file, but the AWS infrastructure (EC2, VPC, S3, etc.) is still running.

Interview Questions

- What is the Terraform state file?
 - What happens if it is deleted?
 - Can Terraform recreate it?
 - How do you recover?
 - How do you prevent this?
-

What is Terraform State?

The **Terraform State** (`terraform.tfstate`) file maintains the mapping between **Terraform configuration** and the **actual infrastructure**.

```
Terraform Code
    ↓
terraform.tfstate
    ↓
AWS Resources
```

It stores:

- Resource IDs
- ARNs
- Current resource state
- Dependencies
- Metadata

Without the state file, Terraform doesn't know which resources it manages.

What Happens if the State File is Deleted?

If you run:

```
terraform plan
```

Terraform assumes no infrastructure exists and tries to recreate everything.

Example:

```
+ Create VPC
+ Create EC2
+ Create ALB
```

This may lead to:

- Duplicate resources
- Deployment failures
- Infrastructure inconsistency

Recovery Steps

1. Verify Existing Infrastructure

```
aws ec2 describe-instances
aws s3 ls
```

2. Check Remote Backend

If using an S3 backend:

```
terraform init
```

Terraform downloads the latest state automatically.

3. Restore or Import State

If no backup exists:

```
terraform import aws_instance.web i-123456789
```

Import all existing resources into the state.

4. Validate

```
terraform plan
```

Expected output:

No changes.
Infrastructure matches configuration.

Common Causes

Cause	Solution
State file deleted	Restore backup or import resources
State corruption	Recover previous version
Multiple local state files	Use remote backend
Accidental overwrite	Enable S3 Versioning

Best Practices

- Never use local state in production.
 - Store state in **S3**.
 - Enable **Versioning** and **KMS encryption**.
 - Use **DynamoDB** for state locking.
 - Restrict access with IAM.
-

Production Example

A developer accidentally deleted the local state file after creating an EKS cluster.

Since the project used an **S3 backend with versioning**, running:

```
terraform init
```

downloaded the latest state, and no infrastructure changes were required.

Interview Tips

Q. Can Terraform recreate the state automatically?

No. You must restore the state from the backend/backup or import existing resources.

Q. Why is a Remote Backend recommended?

It provides:

- Centralized state
- Team collaboration
- Backup & Versioning
- Encryption
- State locking

Q. What is the recommended production setup?

```
Terraform
  ↓
S3 Backend (State File)
  ↓
Versioning + KMS
  ↓
DynamoDB (State Locking)
```

Scenario 2: Two Engineers Run `terraform apply` Simultaneously

Scenario

Two DevOps engineers run:

```
terraform apply
```

at the same time on the same infrastructure.

Interview Questions

- What happens?
 - Why is it dangerous?
 - How does Terraform prevent this?
-

Problem

Both engineers try to update the same **Terraform state**, which can lead to:

- State corruption
 - Infrastructure inconsistency
 - Failed deployments
-

Solution: State Locking

Terraform locks the state before making changes.

```
terraform apply
```

Output:

```
Acquiring state lock...
```

If another user runs `terraform apply` simultaneously:

```
Error acquiring the state lock
```

The second user must wait until the first operation completes.

How Does It Work?

```
Terraform
  ↓
S3 (State File)
  ↓
DynamoDB (State Lock)
```

- **S3** stores the state file.
 - **DynamoDB** stores the lock.
 - Only one user can modify the state at a time.
-

Recovery

If a lock remains after an interrupted deployment:

```
terraform force-unlock <LOCK_ID>
```

Use only after confirming no Terraform process is running.

Best Practices

- Use S3 + DynamoDB backend.
- Avoid local state.
- Run Terraform through CI/CD pipelines.
- Never delete locks manually.

Production Example

Two Jenkins pipelines started simultaneously. One acquired the lock, while the second failed with a state lock error. After the first completed, the second deployment succeeded.

Interview Tips

Q. Why is DynamoDB required?

It provides **state locking**, preventing multiple users from modifying the state simultaneously.

Scenario 3: Manual Changes in AWS Console (Infrastructure Drift)

Scenario

An EC2 instance was created using Terraform, but someone manually changed its Security Group from the AWS Console.

Interview Questions

- What is Infrastructure Drift?
 - How do you detect it?
 - How do you fix it?
-

What is Drift?

Drift occurs when infrastructure is modified **outside Terraform**, causing Terraform configuration and actual infrastructure to differ.

```
Terraform Code
    ↓
AWS Resource Modified Manually
    ↓
Infrastructure Drift
```

Detect Drift

```
terraform plan
```

Example:

```
~ security_group = sg-01 -> sg-02
```

Terraform detects that AWS differs from the state/configuration.

Solutions

- If manual change is incorrect → Run:

```
terraform apply
```

Terraform restores the desired state.

- If manual change should be kept → Update Terraform code first, then apply.
- If resource wasn't managed → Use:

```
terraform import
```

Best Practices

- Avoid manual AWS changes.
 - Restrict console access using IAM.
 - Use CI/CD for infrastructure changes.
 - Run `terraform plan` regularly.
-

Production Example

An engineer manually opened SSH (Port 22) in a Security Group. During the next deployment, Terraform removed the unauthorized rule because it wasn't defined in code.

Interview Tips

Q. Which command detects Drift?

```
terraform plan
```

Scenario 4: Resource Renamed Without Recreation

Scenario

You rename:

```
aws_instance.web
```

to

```
aws_instance.app
```

Terraform now wants to destroy and recreate the EC2 instance.

Interview Questions

- Why?
 - How do you prevent resource recreation?
-

Why Does This Happen?

Terraform tracks resources by **resource address**, not by AWS resource name.

Changing the resource name makes Terraform think:

```
Old Resource → Delete
```

```
New Resource → Create
```

Solution 1 (Recommended)

Use a **moved block**:

```
moved {  
  from = aws_instance.web  
  to   = aws_instance.app  
}
```

Terraform updates the state without recreating the resource.

Solution 2

Move the state manually:

```
terraform state mv aws_instance.web aws_instance.app
```

Best Practices

- Use `moved` blocks when refactoring.
 - Always review `terraform plan`.
 - Never rename production resources without updating the state.
-

Production Example

During module refactoring, developers renamed multiple resources. Using `moved` blocks avoided recreating EC2 instances and prevented downtime.

Interview Tips

Q. Does renaming always recreate resources?

Yes, unless you use a **moved block** or `terraform state mv`.

Scenario 5: `terraform apply` Failed Midway

Scenario

Terraform successfully creates:

- VPC ✓
- Subnets ✓

But EC2 creation fails due to an AWS quota or IAM issue.

Interview Questions

- What happens?
 - Does Terraform rollback?
 - How do you recover?
-

What Happens?

Terraform **does not automatically rollback**.

Resources already created remain in AWS and in the state file.

VPC	✓
Subnet	✓
EC2	✗
ALB	✗

Recovery Steps

1. Check the Error

```
terraform plan
```

or

```
terraform apply
```

Identify the root cause:

- IAM permission
 - Service quota
 - Invalid configuration
 - API timeout
-

2. Fix the Issue

Examples:

- Increase AWS quota.
 - Correct IAM permissions.
 - Fix Terraform code.
-

3. Re-run

```
terraform apply
```

Terraform creates **only the missing resources**.

Best Practices

- Always review `terraform plan`.
 - Use remote state.
 - Test in lower environments.
 - Deploy through CI/CD.
-

Production Example

A deployment failed while creating an ALB because the AWS quota had been reached. After increasing the quota, rerunning `terraform apply` created only the ALB without affecting the existing VPC or Subnets.

Interview Tips

Q. Does Terraform rollback automatically?

No. Terraform follows a **partial apply** model. Successfully created resources remain, and the next `terraform apply` creates only the remaining resources.

Quick Revision (Scenarios 2–5)

Scenario	First Command/Concept	Solution
Simultaneous Apply	State Locking	S3 + DynamoDB
Infrastructure Drift	<code>terraform plan</code>	Update code or apply
Resource Rename	<code>moved block</code>	Prevent recreation
Apply Failed	<code>terraform plan</code>	Fix issue and re-run <code>terraform apply</code>

Scenario 6: Existing AWS Resources Need to be Managed by Terraform

Scenario

Your company already has an EC2 instance created manually from the AWS Console. Now the team wants Terraform to manage it without recreating it.

Interview Questions

- Can Terraform manage existing resources?
 - How do you import them?
 - Will Terraform recreate them?
-

Solution

Terraform **cannot automatically manage existing resources**. First, write the resource block, then import it into the state.

Example:

```
terraform import aws_instance.web i-0123456789
```

Verify:

```
terraform plan
```

If the configuration matches, Terraform should show:

```
No changes.
```

Best Practices

- Import resources before making changes.
 - Match the Terraform configuration with the existing resource.
 - Verify using `terraform plan`.
-

Production Example

The networking team manually created an S3 bucket. Before automation, the DevOps team imported it into Terraform to avoid deleting and recreating the bucket.

Interview Tip

Q. Does `terraform import` create infrastructure?

No. It only adds an existing resource to the Terraform state.

Scenario 7: Managing Multiple Environments (Dev, QA, Prod)

Scenario

Your company has **Dev**, **QA**, and **Production** environments. You need to manage all using Terraform.

Interview Questions

- How do you manage multiple environments?
 - Workspaces vs tfvars?
 - Which approach is better?
-

Common Approaches

Option 1 (Recommended)

Use separate backend and variable files.

```
dev.tfvars
```

```
qa.tfvars
```

```
prod.tfvars
```

Deploy:

```
terraform apply -var-file=prod.tfvars
```

Option 2

Terraform Workspaces

```
terraform workspace new dev
```

```
terraform workspace new prod
```

Best Practices

- Separate backend for each environment.
- Separate state files.
- Use modules for code reuse.
- Avoid workspaces for large production environments.

Production Example

The company used:

- Same Terraform modules
- Different `tfvars`
- Separate S3 backend for each environment

This isolated Dev, QA, and Production completely.

Interview Tip

Q. Workspaces or `tfvars`?

For enterprise projects, **`tfvars` + separate backend** is generally preferred because it provides better isolation and reduces operational risk.

Scenario 8: Reusable Terraform Modules

Scenario

Multiple teams need identical VPCs, EC2 instances, and Security Groups.

Interview Questions

- What is a module?
 - How do you make modules reusable?
 - Why use modules?
-

Solution

Create reusable modules.

```
modules/
```

```
  vpc/
```

```
  ec2/
```

```
  alb/
```

Use:

```
module "vpc" {  
  source = "../modules/vpc"  
}
```

Benefits

- Reusable code
 - Easy maintenance
 - Consistent infrastructure
 - Reduced duplication
-

Best Practices

- Keep modules small and focused.
 - Use variables and outputs.
 - Version modules using Git tags.
 - Avoid hardcoding values.
-

Production Example

Instead of maintaining 30 different VPC configurations, the team created one reusable VPC module used across all projects.

Interview Tip

Q. What makes a good Terraform module?

- Reusable
 - Parameterized
 - Version controlled
 - Independent
 - Well documented
-

Scenario 9: Managing Secrets Securely

Scenario

Your application requires a database password. Developers want to store it directly in Terraform code.

Interview Questions

- Is this a good practice?
 - Where should secrets be stored?
 - Does Terraform state contain secrets?
-

Wrong Approach

```
password = "Admin123"
```

Never hardcode secrets.

Recommended Solution

Use:

- AWS Secrets Manager
- AWS Systems Manager Parameter Store

Example:

```
data "aws_secretsmanager_secret_version" "db" {}
```

Mark variables as sensitive:

```
variable "db_password" {  
  sensitive = true  
}
```

Best Practices

- Never commit secrets to Git.
 - Encrypt Terraform state.
 - Use IAM to restrict access.
 - Rotate secrets regularly.
-

Production Example

Database passwords were stored in AWS Secrets Manager. Terraform only referenced the secret ARN, ensuring passwords never appeared in the code.

Interview Tip

Q. Does Terraform State contain secrets?

Yes. Unless properly protected, secrets may be stored in the state file. Secure the backend with encryption and restricted access.

Scenario 10: Recreate Only One Resource

Scenario

One EC2 instance is corrupted, but the rest of the infrastructure is healthy.

Interview Questions

- How do you recreate only one resource?
 - Can Terraform apply changes to a single resource?
-

Option 1 (Preferred)

Replace the resource:

```
terraform apply -replace="aws_instance.web"
```

Terraform destroys and recreates only that resource.

Option 2

Target a specific resource:

```
terraform apply -target="aws_instance.web"
```

Use `-target` carefully. It is mainly for troubleshooting and exceptional situations.

Best Practices

- Prefer `-replace` over `-target` for resource recreation.
 - Always run `terraform plan` before applying.
 - Avoid routine use of `-target`.
-

Production Example

A production EC2 instance became unreachable due to OS corruption. The team used:

```
terraform apply -replace="aws_instance.web"
```

Only that instance was recreated while the rest of the infrastructure remained unchanged.

Interview Tip

Q. Difference between `-replace` and `-target`?

- `-replace` → Recreates an existing resource.
 - `-target` → Applies changes only to a selected resource or module and should be used sparingly.
-

Quick Revision (Scenarios 6–10)

Scenario	First Command/Concept	Solution
Import Existing Resources	<code>terraform import</code>	Import into state
Multiple Environments	<code>tfvars</code> + Separate Backend	Environment isolation
Reusable Modules	<code>module</code> block	Code reuse
Secrets Management	AWS Secrets Manager/SSM	Avoid hardcoding
Recreate One Resource	<code>terraform apply -replace</code>	Replace only the affected resource

Most Asked Follow-up Questions

Question	Answer
Can Terraform manage existing resources?	Yes, using <code>terraform import</code> .
What is the preferred way to manage multiple environments?	Separate backends with different <code>tfvars</code> files.

Question	Answer
Why use modules?	Reusability, consistency, and maintainability.
Should secrets be stored in Terraform code?	No, use Secrets Manager or Parameter Store.
When should <code>-target</code> be used?	Only for exceptional troubleshooting or recovery scenarios.

Scenario 11: Automating Terraform Using Jenkins/GitHub Actions

Scenario

Your company wants every infrastructure change to happen through Jenkins instead of engineers running Terraform from their laptops.

Interview Questions

- How do you automate Terraform?
 - What stages do you include in the pipeline?
 - How do you prevent unauthorized deployments?
-

CI/CD Flow

```
Git Commit
  ↓
Jenkins/GitHub Actions
  ↓
terraform fmt
  ↓
terraform validate
  ↓
terraform init
  ↓
terraform plan
  ↓
Manual Approval
  ↓
terraform apply
```

Best Practices

- Store state in S3.
- Use DynamoDB for locking.
- Keep AWS credentials in Jenkins Credentials/Secrets Manager.
- Require approval before Production deployment.

Production Example

Every Pull Request automatically runs:

```
terraform fmt
```

```
terraform validate
```

```
terraform plan
```

Only after approval does Jenkins execute:

```
terraform apply
```

Interview Tip

Q. Why keep manual approval before Production?

To review infrastructure changes and avoid accidental resource deletion.

Scenario 12: Upgrading Terraform Version

Scenario

Your project uses Terraform **1.4**, but the organization wants to upgrade to **1.8**.

Interview Questions

- How do you upgrade safely?
 - What should you verify?
-

Upgrade Steps

1. Read Terraform release notes.
2. Backup the state file.
3. Upgrade Terraform version.
4. Upgrade providers.

```
terraform init -upgrade
```

5. Run:

```
terraform plan
```

6. Test in Dev before Production.

Common Issues

Issue	Solution
Provider incompatibility	Upgrade provider
Deprecated syntax	Update code
State migration	Backup state first

Best Practices

- Upgrade gradually.
- Test in lower environments.
- Pin Terraform and provider versions.

Example:

```
terraform {  
  required_version = "~> 1.8"  
}
```

Production Example

During an upgrade, the AWS provider introduced deprecated arguments. The team updated the Terraform code in Dev, validated it, and then upgraded Production.

Interview Tip

Q. What is the first step before upgrading?

Always **backup the Terraform state**.

Scenario 13: Shared Module Used by Multiple Projects

Scenario

Your VPC module is used by **20 different projects**. A new feature needs to be added without breaking existing deployments.

Interview Questions

- How do you update shared modules safely?
 - How do you version modules?
-

Solution

Version your modules using Git tags.

Example:

v1.0

v1.1

v2.0

Projects specify the version:

```
module "vpc" {  
  source = "git::https://repo/vpc.git?ref=v1.1"  
}
```

Best Practices

- Never modify production modules directly.
 - Use semantic versioning.
 - Test module changes before release.
 - Upgrade projects gradually.
-

Production Example

The networking team released **VPC Module v2.0**. Existing projects continued using **v1.0** until testing was completed, avoiding production outages.

Interview Tip

Q. Why version Terraform modules?

To prevent breaking changes from affecting all projects at once.

Scenario 14: Migrating Local Backend to S3 Backend

Scenario

Terraform currently stores state locally. The company wants to move it to an S3 backend.

Interview Questions

- How do you migrate the backend?
 - Will infrastructure be recreated?
-

Migration Steps

Configure backend:

```
terraform {  
  backend "s3" {  
    bucket = "terraform-state"  
    key    = "prod/terraform.tfstate"  
    region = "ap-south-1"  
  }  
}
```

Run:

```
terraform init
```

Terraform asks:

```
Do you want to copy existing state?
```

Choose:

```
yes
```

The local state is migrated to S3.

Best Practices

- Enable Versioning.
 - Enable KMS encryption.
 - Configure DynamoDB locking.
 - Verify backend after migration.
-

Production Example

A company migrated from local state to S3 without recreating any resources. Terraform simply copied the existing state file to the remote backend.

Interview Tip

Q. Does backend migration recreate resources?

No. Only the state location changes.

Scenario 15: Cross-Account AWS Deployment

Scenario

Your Jenkins server is in **AWS Account A**, but infrastructure needs to be deployed into **AWS Account B**.

Interview Questions

- How do you deploy across AWS accounts?
 - How does Terraform authenticate?
-

Solution

Use **IAM AssumeRole**.

Terraform Provider:


```
provider "aws" {  
  region = "ap-south-1"  
  
  assume_role {  
    role_arn = "arn:aws:iam::ACCOUNT_B:role/TerraformRole"  
  }  
}
```

Flow:

```
Jenkins  
  ↓  
Account A  
  ↓  
Assume IAM Role  
  ↓  
Account B  
  ↓  
Deploy Resources
```

Best Practices

- Use IAM Roles instead of Access Keys.
 - Grant least privilege permissions.
 - Use provider aliases if deploying to multiple accounts.
-

Production Example

A centralized Jenkins pipeline assumed IAM roles in multiple AWS accounts and deployed infrastructure without storing permanent AWS credentials.

Interview Tip

Q. Why use AssumeRole instead of Access Keys?

- More secure
 - Temporary credentials
 - Easier access management
 - No hardcoded secrets
-

Quick Revision (Scenarios 11–15)

Scenario	First Command/Concept	Solution
CI/CD Pipeline	<code>terraform plan</code>	Approval before Apply
Terraform Upgrade	<code>terraform init -upgrade</code>	Backup state first
Module Versioning	Git Tags	Semantic Versioning
Backend Migration	<code>terraform init</code>	Move state to S3
Cross-Account Deployment	IAM AssumeRole	Provider AssumeRole

Most Asked Follow-up Questions

Question	Answer
What are the stages in a Terraform pipeline?	<code>fmt</code> → <code>validate</code> → <code>init</code> → <code>plan</code> → <code>approval</code> → <code>apply</code>
Should Production deploy automatically?	Usually No , keep manual approval.
Does backend migration recreate resources?	No, only the state location changes.
How do you manage shared modules?	Version them using Git tags.
How do you deploy to another AWS account?	Use IAM AssumeRole with the AWS provider.

Scenario 16: Terraform Execution is Very Slow

Scenario

Your Terraform deployment, which normally takes 5 minutes, is now taking 25–30 minutes.

Interview Questions

- Why is Terraform slow?
 - How do you improve performance?
 - What do you check first?
-

Troubleshooting

Check:

- Large state file
- Too many resources in one project
- API throttling

- Slow provider
- Module dependencies

Useful commands:

```
terraform plan
```

```
terraform apply -parallelism=20
```

Common Causes

Cause	Solution
Large state file	Split infrastructure into modules
Too many resources	Separate projects (Networking, Compute, DB)
API throttling	Reduce parallel requests or retry
Unnecessary dependencies	Remove unwanted <code>depends_on</code>

Best Practices

- Keep state files small.
 - Separate infrastructure into modules.
 - Avoid unnecessary `depends_on`.
 - Use remote backend.
-

Production Example

A single Terraform project managed over **1,500 AWS resources**. The team split it into separate modules (Network, EKS, RDS), reducing deployment time from **30 minutes to 8 minutes**.

Interview Tip

Q. How do you optimize Terraform?

Split large projects, reduce dependencies, and optimize module design.

Scenario 17: Resources Created in Wrong Order

Scenario

Terraform tries to create an EC2 instance before its Security Group or IAM Role exists, causing deployment failures.

Interview Questions

- How does Terraform determine resource order?
 - When should you use `depends_on`?
-

Solution

Terraform automatically creates an **implicit dependency** when one resource references another.

Example:

```
vpc_security_group_ids = [aws_security_group.web.id]
```

If no reference exists, use:

```
depends_on = [  
    aws_iam_role.ec2_role  
]
```

Best Practices

- Prefer implicit dependencies.
 - Use `depends_on` only when necessary.
 - Avoid excessive manual dependencies.
-

Production Example

An EC2 instance required an IAM Role before startup. Adding `depends_on` ensured Terraform created the IAM Role first.

Interview Tip

Q. Which is better: Implicit or Explicit dependency?

Implicit dependency is preferred because Terraform manages it automatically.

Scenario 18: Blue-Green Infrastructure Deployment

Scenario

The company wants to deploy a new application version without downtime.

Interview Questions

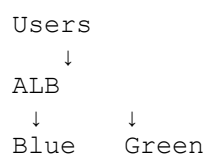
- How do you implement Blue-Green deployment using Terraform?
 - How is traffic switched?
-

Solution

Terraform creates:

- Blue Environment (Current)
- Green Environment (New)

Flow:



After validation:

- Update ALB Target Group
- Or update Route53 record

Traffic shifts to Green.

Best Practices

- Test Green before switching traffic.
 - Keep Blue until rollback is no longer needed.
 - Automate traffic switching.
-

Production Example

During a banking application upgrade, Terraform created a new Auto Scaling Group. After health checks passed, the ALB redirected traffic to the new environment with zero downtime.

Interview Tip

Q. How do you rollback?

Point the ALB or Route53 back to the Blue environment.

Scenario 19: Securing Terraform State File

Scenario

Terraform state contains database passwords and sensitive infrastructure information.

Interview Questions

- Why is the state file sensitive?
 - How do you secure it?
-

Why Secure the State?

State may contain:

- Passwords
- Secret values
- Resource IDs
- Database endpoints

Anyone with access to the state file can potentially access your infrastructure.

Best Practices

- Store state in **S3**.
- Enable **KMS encryption**.
- Enable **Versioning**.
- Use **DynamoDB** for locking.
- Restrict access using IAM.

Architecture:

```
Terraform
  ↓
S3 Backend
  ↓
KMS Encryption
  ↓
Versioning
  ↓
DynamoDB Lock
```

Production Example

The organization restricted access to the Terraform state bucket using IAM roles. Only the Jenkins pipeline and DevOps administrators could read or modify the state.

Interview Tip

Q. Should Terraform state be stored in Git?

Never. It may expose sensitive infrastructure information and secrets.

Scenario 20: Production Terraform Deployment Failure

Scenario

A production deployment fails:

- VPC created ✓
- Subnets created ✓
- ALB failed ✗
- Application unavailable

Interview Questions

- How do you recover?
 - What is your troubleshooting approach?
-

Recovery Steps

1. Review the Error

```
terraform plan
```

```
terraform apply
```

Identify:

- IAM permission
 - AWS quota
 - Invalid configuration
 - Provider error
-

2. Verify State

Check whether successful resources exist in the state.

```
terraform state list
```

3. Fix the Root Cause

Examples:

- Increase AWS quota.
 - Update IAM permissions.
 - Correct Terraform code.
-

4. Re-run

```
terraform apply
```

Terraform creates only the missing resources.

5. Validate

Verify:

- Infrastructure
- Application health
- Monitoring alerts

Perform RCA after service restoration.

Best Practices

- Always review `terraform plan`.
 - Store state remotely.
 - Use CI/CD with approval.
 - Test changes in Dev before Production.
-

Production Example

A deployment failed because the ALB quota was exceeded. After requesting a quota increase, rerunning `terraform apply` created only the ALB while leaving the existing VPC and Subnets untouched.

Interview Tip

Q. What should be your first priority during a Terraform incident?

1. Restore service.
 2. Identify root cause.
 3. Validate infrastructure.
 4. Perform Root Cause Analysis (RCA).
 5. Implement preventive measures.
-

Quick Revision (Scenarios 16–20)

Scenario	First Command/Concept	Solution
Slow Terraform	<code>terraform plan</code>	Split modules, optimize state
Wrong Resource Order	Implicit dependency	Use <code>depends_on</code> if needed

Scenario	First Command/Concept	Solution
Blue-Green Deployment	ALB / Route53	Switch traffic after validation
Secure State	S3 + KMS + DynamoDB	Encrypt and restrict access
Production Failure	<code>terraform state list</code>	Fix issue and re-run <code>apply</code>

Terraform Interview Cheat Sheet

Scenario	First Command / Focus
State Deleted	<code>terraform init</code> / Restore state
State Lock	DynamoDB
Drift Detection	<code>terraform plan</code>
Resource Rename	<code>moved</code> block
Apply Failed	<code>terraform plan</code>
Import Resources	<code>terraform import</code>
Multiple Environments	<code>tfvars</code> + Separate Backend
Modules	Reusable modules
Secrets	AWS Secrets Manager
Single Resource Replace	<code>terraform apply -replace</code>
CI/CD	<code>plan</code> → Approval → <code>apply</code>
Upgrade	<code>terraform init -upgrade</code>
Module Versioning	Git Tags
Backend Migration	<code>terraform init</code>
Cross Account	IAM AssumeRole
Performance	Small state & modules
Dependencies	Implicit > <code>depends_on</code>
Blue-Green	ALB / Route53
Secure State	S3 + KMS + DynamoDB
Production Incident	Fix root cause → <code>terraform apply</code>

Common Senior-Level Follow-up Questions

- Why use S3 + DynamoDB instead of local state?**
 - Shared state, locking, backup, encryption, collaboration.
- What is the biggest risk in Terraform?**
 - Losing or corrupting the **state file**.
- Can Terraform automatically rollback failed deployments?**
 - No.** It uses a partial apply model.
- What is the difference between `terraform refresh`, `plan`, and `apply`?**
 - refresh:** Updates state from real infrastructure (behavior now integrated into `plan/apply` in modern Terraform).
 - plan:** Shows proposed changes.

- **apply:** Executes the changes.
 - 5. **What is the most important Terraform concept for production?**
 - **Terraform State Management**, because every operation depends on an accurate and protected state file.
-